

ArcAssist-Web Bot: A Prototype Instantiation for Evidence-Grounded Architectural Decision Support

Vanusa Rodrigues de Oliveira
Federal University of Lavras
São Sebastião do Paraíso, Minas
Gerais, Brazil
vanusa.oliveira1@estudante.ufla.br

Bento Rafael Siqueira
Federal University of Lavras
Lavras, Minas Gerais, Brazil
bento.siqueira@ufla.br

Samira Santos da Silva
Federal University of Lavras
São Sebastião do Paraíso, Minas
Gerais, Brazil
samirasilva@ufla.br

Abstract

LLM-based bots can draft plausible software-architecture advice without exposing its evidence or readiness for human review. This paper presents ArcAssist-Web Bot, an early Web-system prototype for evidence-grounded architectural decision preparation. A user states an Evolution Concern; the implemented Flask/RAG path retrieves PDF/text architecture knowledge, constrains an external LLM request, and drafts review material. We map this path and its planned extensions to seven responsibilities: Interaction, Web-System and Domain Context, Context and Knowledge, Prompt Orchestration, LLM Reasoning, Validation and QA, and Knowledge Management. These responsibilities are an author-constructed organizing lens, not a validated taxonomy. A public-form scenario illustrates how domain boundaries, privacy, validation, and architectural knowledge enter a structured Decision Package before an external Architect Review gate. The artifact inspection supports technical feasibility of the implemented RAG path, not workflow effectiveness. The contribution is a concrete prototype instantiation, an explicit boundary between drafting and decision acceptance, and an evaluation agenda for traceability, usefulness, and review effort.

Keywords

Software Engineering Bots, Conversational Agents, Retrieval Augmented Generation, Architectural Decision Support

1 Introduction

Software Engineering bots automate or mediate activities from repository maintenance to conversational support and autonomous execution [3, 15]. With LLMs, they can interpret requests, retrieve evidence, synthesize alternatives, and explain consequences. This is attractive for Web evolution, where small changes may cross flows, APIs, domain rules, deployment constraints, privacy, and quality attributes [5–7, 11].

Fluent output also creates a design risk: users may confuse retrieved evidence, model inference, and an accepted architectural decision. In our running concern, adding an interest form to a public animal profile appears local but raises consent, duplicate-submission, staff-visibility, API, and domain-boundary decisions. A decision-support bot should expose the concern, evidence, alternatives, assumptions, and required review. Our position is therefore that the bot prepares decision material; it does not decide.

This paper presents ArcAssist-Web Bot, an early prototype instantiation of that position. Its implemented slice covers Web interaction, architecture-document retrieval, prompt construction, and

LLM drafting. Planned Validation and QA and Knowledge Management turn drafts into inspectable Decision Packages and retain only architect-reviewed outcomes.

The contributions are: (i) a decision-preparation framing; (ii) a prototype view separating implemented RAG functionality from planned review responsibilities; and (iii) an evaluation and artifact strategy. Relative to prior SE bots and architecture-knowledge tools, the novelty is the concrete integration of a RAG-backed prototype, a Decision Package boundary, and an Architect Review gate. Neither RAG nor the individual responsibilities is claimed as new; the seven-responsibility lens only organizes this Web-system prototype and its planned workflow.

2 Background and Related Work

Bots interact with developers, repositories, and project artifacts to support development workflows [3, 15]. Recent WBOTS work shows that LLM-based bots need engineering beyond conversation: requirements templates delimit persona and autonomy [17]; touchpoint models and UX studies evaluate interaction journeys [12, 13, 16]; privacy middleware controls sensitive data [18]; and generated-code and OpenAPI studies require validation and human inspection [10, 20].

Architecture decisions record rationale, alternatives, trade-offs, constraints, and evolving knowledge [6, 11]. RAG conditions answers on external evidence [4, 8], while architecture-support studies use generative AI to retrieve, explain, and organize knowledge but still require verification and accountability [1, 2, 9, 14, 19].

Taken together, these lines of work motivate an inspectable handoff from evidence retrieval to architectural review: retrieved material should remain distinguishable from model inference, generated alternatives should expose unresolved trade-offs, and acceptance should remain a human responsibility. ArcAssist-Web makes this pre-decision path explicit without replacing ADR practices or proposing a new retrieval method.

ArcAssist-Web differs in its integration boundary rather than in a new retrieval algorithm: evidence grounds a draft, a structured package exposes unresolved trade-offs and validation, an architect gates acceptance, and only reviewed outcomes may be retained. This positions the artifact as a prototype of a pre-decision workflow, not an autonomous recommender.

3 Bot Responsibilities

We constructed seven responsibilities through an author-led walk-through: trace the concern through code and data flow, map observable stages to architectural-decision artifacts, and classify each stage as implemented, partial, or planned while replaying the scenario.

The result is a qualitative organizing lens, not a systematically derived or validated taxonomy; it prevents planned capabilities from being counted as current evidence.

Interaction frames the concern; *Web-System and Domain Context* relates it to flows, boundaries, privacy, and quality attributes; *Context and Knowledge* retrieves baseline evidence; *Prompt Orchestration* combines concern and evidence; and *LLM Reasoning* drafts alternatives, consequences, assumptions, risks, and rationale. *Validation and QA* checks traceability and consistency. *Knowledge Management* stores only reviewed ADRs, evidence links, diagrams, and related knowledge. Architect Review remains an external acceptance gate. A Decision Package is the boundary object: a structured record of the concern, versioned evidence identifiers, alternatives, consequences, assumptions, risks, and validation actions; it contains no accepted decision before review.

The boundaries among these responsibilities are intentional. Interaction captures what the user wants to change, whereas Web-System and Domain Context asks what parts of the system and domain may be affected. Context and Knowledge then supplies retrievable material without treating retrieval as a decision. Prompt Orchestration defines how the concern and retrieved evidence are presented to the model and which behaviors are constrained; LLM Reasoning is limited to drafting candidate material. The two planned responsibilities begin where free-form generation should stop: Validation and QA should make evidence references, assumptions, and missing checks explicit, while Knowledge Management should operate only after Architect Review. This separation lets the prototype expose which stages are executable today and which remain part of the research agenda.

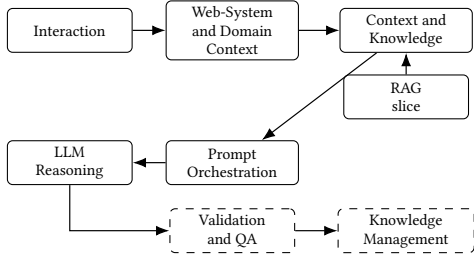


Figure 1: Bot instantiation over architecture-support responsibilities. RAG is the implemented slice inside Context and Knowledge. Planned Validation and QA materializes the Decision Package reviewed before knowledge retention.

Figure 1 separates the current pre-review path from planned package validation and knowledge retention. The bot prepares material; an architect accepts, rejects, or revises the decision. Figure 2E illustrates package checking, and panel F places retention after that human gate.

4 Prototype and RAG Demonstration

Figure 2 distinguishes implemented and proposed interface states. The artifact implements the path from concern submission through PDF/text retrieval and constrained LLM drafting; Validation and QA and Knowledge Management remain planned.

Implementation. A Flask frontend sends a question to /ask and streams the answer. The backend loads a FAISS index,

chunks PDF/text documents with overlap, embeds them with all-MiniLM-L6-v2, retrieves nearest chunks, and injects them into a constrained prompt for an external LLM. A future evidence manifest should record source identifier, title, origin, version, license, and redistribution status.

Request-to-review flow. The implemented request path begins with an Evolution Concern expressed through the Web interface. The concern provides the immediate task and, together with the corpus, the available domain context. Retrieval then selects nearby architecture chunks from the FAISS index; these chunks are passed as context rather than silently merged with the model’s own knowledge. Prompt construction constrains the model to reason from that material, compare alternatives and trade-offs, expose assumptions, and signal when the supplied context is insufficient. The streamed response is therefore treated as a draft for review. The proposed continuation converts that draft into a Decision Package whose evidence identifiers and required validation actions can be checked before an architect accepts, revises, or rejects any decision. Keeping this continuation outside the current implementation is important to our claims: the artifact demonstrates the retrieval-to-draft path, whereas package validation and post-review retention are design targets.

Table 1: Implemented RAG trace and related responsibilities.

Prototype step	Responsibility	Current support
Submit concern	Interaction; Web-System and Domain Context	Implemented through templates/ and /ask; domain context comes mainly from the concern and corpus.
Retrieve evidence	Context and Knowledge	Implemented in rag/vectorstore.py; retrieves chunks but lacks stable source metadata.
Build prompt	Prompt Orchestration	Implemented in rag/llm.py; constrains role, grounding, trade-offs, and insufficient-context behavior.
Draft answer	LLM Reasoning	Implemented through the model adapter; produces review material, not an accepted decision.
Check and retain	Validation and QA; Knowledge Management	Planned; should validate identifiers, unsupported claims, package schema, and accepted ADR updates.

Running demonstration. The concern is to let a visitor register initial interest from a public animal profile without implementing adoption. Retrieval searches for component boundaries, decisions, quality attributes, privacy, and trade-offs. The prompt requests grounded alternatives and explicit assumptions. A draft can compare extending the public-profile module with introducing a bounded *AdoptionInterest* concept, exposing consent, duplicate submissions, staff visibility, API contracts, and ADR updates as validation needs.

The scenario also illustrates why the Decision Package is more than a formatted answer. Evidence identifiers should let a reviewer return to the retrieved material; alternatives and consequences make trade-offs inspectable; assumptions separate model inference from corpus-supported facts; risks and validation actions identify work that must occur before acceptance. In this way, the package is intended to preserve review context across the issue, pull-request, or ADR boundary without treating the LLM output itself as architecture knowledge.

The next step is a structured Decision Package followed by Architect Review. In a regular workflow, an issue supplies the concern;

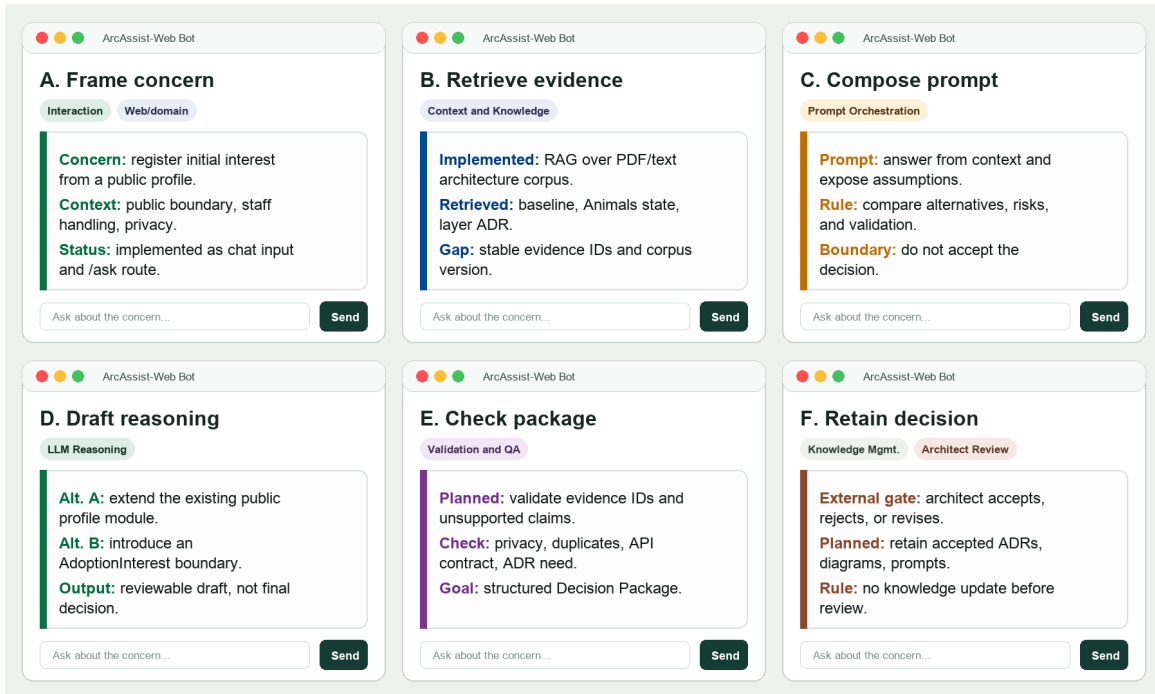


Figure 2: Illustrative screen sequence for the seven responsibilities. The artifact implements RAG-backed chat, retrieval, prompt, and drafting; package checking and reviewed retention are proposed states.

the package accompanies pull-request or ADR review; the architect accepts, revises, or rejects it; and only accepted outcomes enter the knowledge base.

5 Proof-of-Concept Inspection

This proof-of-concept inspection was an author-led structured walk-through of source code, data flow, prompts, and interface states. It involved no users, independent evaluators, baseline comparison, or repeated model trials; its evidence is limited to prototype feasibility.

First, framing the artifact as decision preparation makes its boundary clearer than a generic-chatbot framing: Figures 1–2 and Table 1 separate concern submission, retrieval, prompting, drafting, planned package checks, and reviewed retention.

Second, the RAG slice demonstrates a connected technical path, not usefulness or effectiveness. It retrieves architecture text and grounds prompt construction, but chunks lack stable document, page, section, timestamp, and corpus-version metadata. Reviewers therefore cannot yet inspect evidence with decision-grade precision.

Third, the next implementation target is a structured package rather than longer prose. It must expose the concern, evidence, alternatives, consequences, assumptions, risks, and validation needs, consistent with prior findings that LLM outputs require post-processing, safeguards, and human inspection [10, 18, 20].

Evaluation plan. The next evaluation is organized around three questions. First, *traceability*: can a generated package point to known evidence identifiers and distinguish corpus-supported statements from assumptions? Automated checks can measure identifier validity, missing evidence links, schema completeness, and unsupported-claim candidates over controlled corpora. Second,

decision-preparation quality: does the package surface multiple plausible alternatives, relevant consequences, risks, and concrete validation actions for the scenario? These properties can be scored against scenario-specific expectations without claiming that one architecture is universally correct. Third, *review cost and usefulness*: compared with keyword search and generic prompting, does the structured package help an architect or experienced developer inspect the available evidence and reach a review outcome with less effort or fewer clarification cycles? A human study can record review time, requested revisions, missing trade-offs, validation adequacy, and perceived usefulness. These measures target the pre-decision workflow rather than the quality of a final architectural decision, preserving the boundary of the current contribution.

Evaluation will first automate schema validity, known evidence identifiers, fact/assumption separation, multiple alternatives, and validation actions across controlled corpora. Architects or experienced developers will then compare scenario packages with keyword search and generic prompting, judging usefulness, trade-off clarity, review effort, and validation adequacy. Interaction-journey measures draw on WBOTS touchpoint and UX work [12, 13, 16].

6 Limitations and Artifact Strategy

The knowledge base is illustrative rather than a complete Web-system baseline, and the index lacks stable source metadata. No controlled user study or comparison with keyword search, generic prompting, or alternative bot architectures has been conducted. Thus, current evidence supports technical feasibility and responsibility separation, not improved decision quality, traceability, or review effort. A competitive evaluation requires redistributable

ADRs, diagrams, backlog items, API contracts, deployment constraints, privacy policies, and quality scenarios.

The inspection exercises one running concern and does not establish generality across Web systems, architectural styles, or domains. Because we did not conduct repeated model trials, model-version and generation variability are outside the present evidence. The current corpus is also insufficient for fine-grained provenance: until document, section, timestamp, and corpus-version identifiers are stable, reviewers can inspect retrieved text but cannot reproduce a decision package at the level expected for durable architecture knowledge. These limitations motivate the artifact and evaluation extensions above.

7 Conclusion

This paper presented ArcAssist-Web Bot, an early prototype for evidence-grounded architectural decision preparation. The implemented path combines interaction, domain context, retrieval, prompt orchestration, and LLM drafting; planned Validation and QA and Knowledge Management should produce inspectable packages and retain only reviewed outcomes.

Future work will add stable evidence identifiers and corpus versions, prompt contracts separating evidence from assumptions, automated traceability checks, and reviewed ADR retention. The seven responsibilities are an organizing lens rather than a new framework, and our claims stop at prototype feasibility and responsibility separation; effectiveness awaits comparative and user evaluations.

Artifact Availability

The replication package is available at <https://github.com/californi/arcassist-web-bot>. It includes prototype source code, setup instructions, dependencies, a sanitized sample corpus, and inventory and anonymization-check scripts. Private data and non-redistributable documents were excluded.

Acknowledgments

The authors gratefully acknowledge the financial support provided by the Fundação de Amparo à Pesquisa do Estado de Minas Gerais (FAPEMIG), Brazil, through projects CIN II APQ-06513-25 and CEX-BIS-00192-25.

References

- [1] R. Dhar, K. Vaidhyanathan, and V. Varma. 2024. Leveraging Generative AI for Architecture Knowledge Management. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C 2024)*. IEEE, Piscataway, NJ, USA, 163–166. doi:10.1109/ICSA-C63560.2024.00034
- [2] T. Eisenreich, S. Speth, and S. Wagner. 2024. From Requirements to Architecture: An AI-Based Journey to Semi-Automatically Generate Software Architectures. In *Proceedings of the 1st International Workshop on Designing Software (Designing 2024)*. Association for Computing Machinery, New York, NY, USA, 52–55. doi:10.1145/3643660.3643942
- [3] L. Erlenhov, F. G. de Oliveira Neto, and P. Leitner. 2020. An Empirical Study of Bots in Software Development: Characteristics and Challenges from a Practitioner's Perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 445–455. doi:10.1145/3368089.3409680
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997. arXiv:2312.10997 [cs.CL] <https://arxiv.org/abs/2312.10997>
- [5] D. Garlan and M. Shaw. 1994. *An Introduction to Software Architecture*. Technical Report CMU/SEI-94-TR-021. Software Engineering Institute, Carnegie Mellon University. <https://www.sei.cmu.edu/library/an-introduction-to-software-architecture/>
- [6] A. Jansen and J. Bosch. 2005. Software Architecture as a Set of Architectural Design Decisions. In *Proceedings of the 5th Working IEEE/IFIP Conference on Software Architecture (WICSA 2005)*. IEEE, Piscataway, NJ, USA, 109–120. doi:10.1109/WICSA.2005.61
- [7] P. B. Kruchten. 1995. Architectural Blueprints: The “4+1” View Model of Software Architecture. *IEEE Software* 12, 6 (1995), 42–50. doi:10.1109/52.469759
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS 2020, Vol. 33)*. Curran Associates, Inc., Red Hook, NY, USA, 9459–9474.
- [9] Q. Lu, L. Zhu, X. Xu, Z. Xing, S. Harrer, and J. Whittle. 2024. Towards Responsible Generative AI: A Reference Architecture for Designing Foundation Model Based Agents. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C 2024)*. IEEE, Piscataway, NJ, USA, 119–126. doi:10.1109/ICSA-C63560.2024.00028
- [10] C. S. Machado, E. K. Viegas, S. M. Melo, and L. N. Paschoal. 2025. Do LLMs Support Generating Code that is Compliant with Creational Design Patterns?. In *Anais do II Workshop sobre Bots na Engenharia de Software (WBOTS 2025)*. Sociedade Brasileira de Computação, Porto Alegre, RS, Brazil, 31–40. doi:10.5753/wbots.2025.15181
- [11] D. E. Perry and A. L. Wolf. 1992. Foundations for the Study of Software Architecture. *ACM SIGSOFT Software Engineering Notes* 17, 4 (1992), 40–52. doi:10.1145/141874.141884
- [12] B. M. Ribeiro, K. H. S. Souza, L. G. Silva, J. Villa, R. F. Vilela, P. H. D. Valle, and W. Silva. 2025. An Experience Report on User-Centered Chatbot Design through Design Thinking. In *Anais do II Workshop sobre Bots na Engenharia de Software (WBOTS 2025)*. Sociedade Brasileira de Computação, Porto Alegre, RS, Brazil, 57–66. doi:10.5753/wbots.2025.15205
- [13] M. Rocha, P. Soares, L. Caldeira, P. Souza, R. F. Vilela, P. H. D. Valle, and W. Silva. 2025. Tecnologias de Avaliação da Experiência do Usuário (UX) em Chatbots: Um Mapeamento Sistemático da Literatura. In *Anais do II Workshop sobre Bots na Engenharia de Software (WBOTS 2025)*. Sociedade Brasileira de Computação, Porto Alegre, RS, Brazil, 21–30. doi:10.5753/wbots.2025.15180
- [14] R. Rubel, A. Di Salle, and A. Bucaioni. 2025. LLM-Based Recommender Systems for Violation Resolutions in Continuous Architectural Conformance. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C 2025)*. IEEE, Piscataway, NJ, USA, 404–409. doi:10.1109/ICSA-C65153.2025.00063
- [15] E. Shihab, S. Wagner, M. A. Gerosa, M. Wessel, and J. Cabot. 2022. The Present and Future of Bots in Software Engineering. *IEEE Software* 39, 5 (2022), 28–31. doi:10.1109/MS.2022.3176864
- [16] C. H. C. da Silva, A. R. Cárdenas Maita, G. A. de Almeida Filho, C. V. M. da Silva, G. Dimant, R. A. Almeida, G. M. L. Costa, A. M. C. Araujo, B. C. D. S. Santana, E. A. Blay, and S. M. Peres. 2025. Evaluating LLM-Based Chatbots through Touchpoint-Driven Process Models. In *Anais do II Workshop sobre Bots na Engenharia de Software (WBOTS 2025)*. Sociedade Brasileira de Computação, Porto Alegre, RS, Brazil, 1–10. doi:10.5753/wbots.2025.14148
- [17] C. V. M. da Silva, A. R. Cárdenas Maita, G. A. de Almeida Filho, C. H. C. da Silva, G. Dimant, R. A. Almeida, G. M. L. Costa, A. M. C. Araujo, B. C. D. S. Santana, E. A. Blay, and S. M. Peres. 2025. Template-Driven Specification of Requirements for LLM-Based Chatbots. In *Anais do II Workshop sobre Bots na Engenharia de Software (WBOTS 2025)*. Sociedade Brasileira de Computação, Porto Alegre, RS, Brazil, 11–20. doi:10.5753/wbots.2025.14151
- [18] F. D. L. da Silva and T. A. Coletti. 2025. Middleware para Intercepção de Dados Sensíveis em Chatbots com Modelos de Linguagem de Grande Porte (LLMs). In *Anais do II Workshop sobre Bots na Engenharia de Software (WBOTS 2025)*. Sociedade Brasileira de Computação, Porto Alegre, RS, Brazil, 41–46. doi:10.5753/wbots.2025.15215
- [19] M. Soliman and J. Keim. 2025. Do Large Language Models Contain Software Architectural Knowledge? An Exploratory Case Study with GPT. In *2025 IEEE 22nd International Conference on Software Architecture (ICSA 2025)*. IEEE, Piscataway, NJ, USA, 13–24. doi:10.1109/ICSA65012.2025.00012
- [20] R. F. Vilela, S. A. de Andrade, E. B. F. Santos, W. Silva, and P. H. D. Valle. 2025. An Intelligent Agent for Automated Test Generation from OpenAPI Specifications. In *Anais do II Workshop sobre Bots na Engenharia de Software (WBOTS 2025)*. Sociedade Brasileira de Computação, Porto Alegre, RS, Brazil, 67–76. doi:10.5753/wbots.2025.15217